

COS 214 Practical 5

1. System Description and Team Details

1.1 Project Title & Domain Explanation

CampusGuard: Multi-Department Emergency Response Coordination Platform.

Domain Architecture & Business Problem

CampusGuard models an enterprise incident coordination platform managing physical campus emergencies across four operational departments: Security, Medical, Facilities, and Fire Response. The system models physical access hardware, operational dispatching, and dynamic emergency playbooks:

- **Command Encapsulation (Command Pattern):** Operator actions (`PerimeterLockCommand` , `DispatchUnitCommand` , `ApplyStrategyCommand`) are encapsulated as executable objects and stored in an `OperatorConsole` history stack, providing full rollback/undo capability during false alarms.
- **Tactical Playbooks (Strategy Pattern):** Subsystems dynamically select and execute interchangeable tactical algorithms (`FullEvacuationStrategy` vs. `LockdownContainmentStrategy`) tailored to incident types at runtime.
- **Incident Lifecycles (State Pattern):** Emergency incidents transition through strict lifecycle states (`ReportedState` → `DispatchedState` → `ContainedState` → `ResolvedState`). Invalid transitions (e.g., resolving before containment) are blocked with explicit rejection guards.
- **Subsystem Coordination (Mediator Pattern):** Emergency units (`SecurityUnit` , `MedicalTeam` , `FacilitiesUnit` , `FireResponseUnit`) coordinate crisis reactions through a `CentralDispatchMediator` without direct peer-to-peer coupling.
- **Hardware Bridging (Adapter Pattern):** A `TurnstileAdapter` adapts modern high-level string requests into legacy integer sector hashes and bit-flags required by existing campus access control hardware (`LegacyTurnstileSystem`).
- **High-level Workflows (Facade Pattern):** `CampusEmergencyFacade` coordinates macro emergency procedures (e.g. chemical spill containment or general campus evacuation) hiding complex multi-step command sequences behind a unified interface.

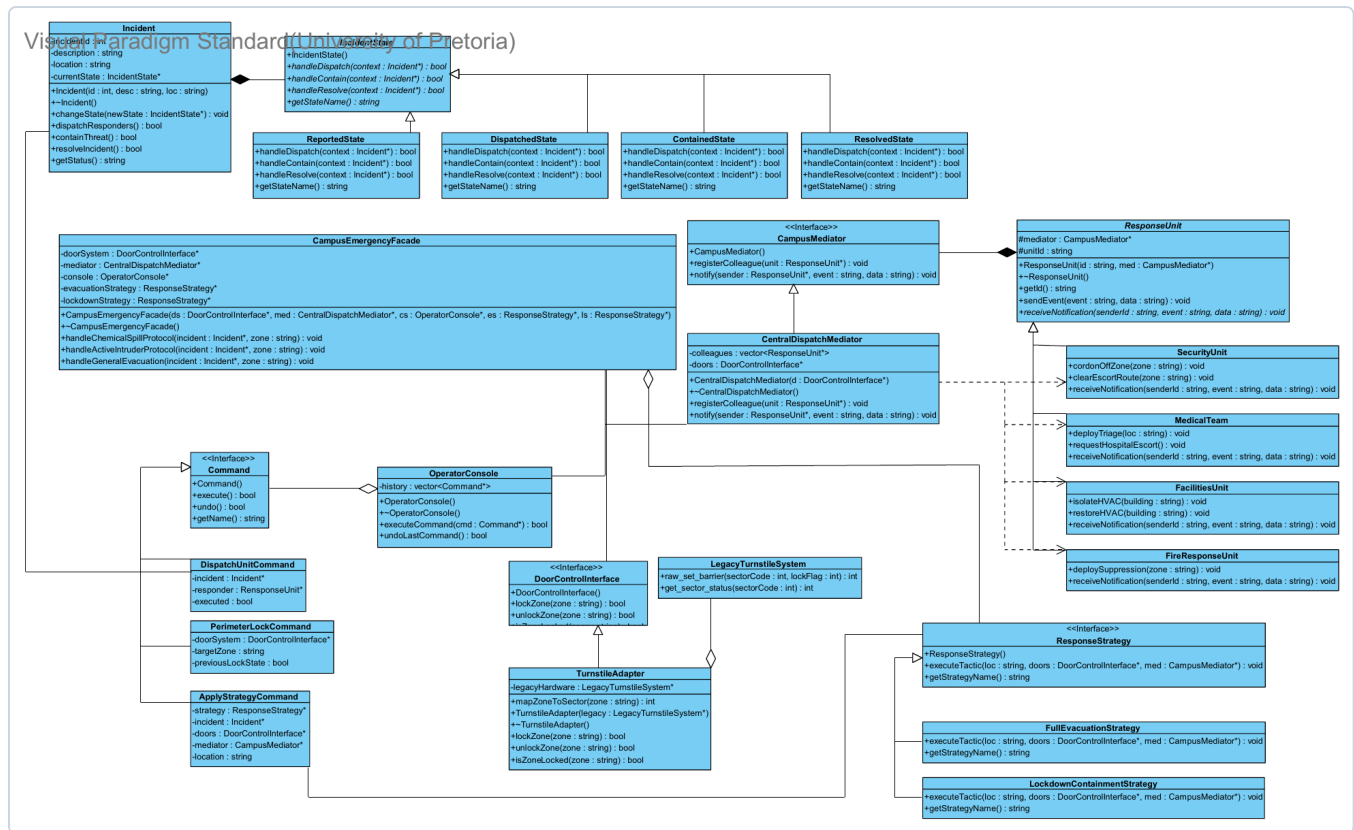
1.2 Team Information

- **1:** Enock Nyamweya (`u25619871`)
- **2:** Kyle Pillay (`u23545756`)
- **3:** Terrence Hluyo (`u25446968`)

1.3 Repository URL

- **GitHub Repository:** <https://github.com/enockNyamweya/campusguard>

The Master UML Class Diagram illustrates the complete CampusGuard architecture, modeling all six GoF design patterns in a unified domain model.



3. GoF Participant Mapping Table

Pattern	GoF Role	System Class Name	Description / Responsibilities
Command	Command	Command	Abstract base interface declaring <code>execute()</code> and <code>undo()</code> operations.
	ConcreteCommand	PerimeterLockCommand , ApplyStrategyCommand , DispatchUnitCommand	Encapsulates parameters, target receivers, and rollback logic for operational campus actions.
	Invoker	OperatorConsole	Maintains execution pipeline and command history stack for undo operations.
Strategy	Strategy	ResponseStrategy	Abstract algorithmic interface declaring <code>executeTactic()</code> .
	ConcreteStrategy	FullEvacuationStrategy , LockdownContainmentStrategy	Implements concrete tactical procedures (unlocking gates & sounding evacuation vs. locking zones & containment).
	Context	ApplyStrategyCommand	Maintains reference to a concrete <code>ResponseStrategy</code> and invokes it polymorphically during command execution.
State	Context	Incident	Maintains current <code>IncidentState*</code> and delegates lifecycle actions to active state.
	State	IncidentState	Abstract interface defining state-dependent lifecycle methods (<code>dispatchResponders()</code> , <code>containThreat()</code> , <code>resolveIncident()</code>).
	ConcreteState	ReportedState , DispatchedState , ContainedState , ResolvedState	Implements lifecycle validation rules; permits valid transitions and rejects invalid operations with descriptive feedback.
Mediator	Mediator	CampusMediator	Abstract mediation interface defining <code>notify()</code> and <code>registerColleague()</code> .
	ConcreteMediator	CentralDispatchMediator	Coordinates communication across response units without direct inter-unit dependencies.
	Colleague	ResponseUnit	Abstract base class for emergency units holding a reference to the mediator.
	ConcreteColleague	SecurityUnit , MedicalTeam , FacilitiesUnit , FireResponseUnit	Dispatches departmental events (e.g. fire alarms, cordons, triage) and reacts to central notifications.
Facade	Facade	CampusEmergencyFacade	Simplifies complex multi-step crisis handling into unified high-level protocols.
Adapter	Target	DoorControlInterface	Modern string-based interface expected by CampusGuard.
	Adapter	TurnstileAdapter	Translates high-level zone strings into numeric sector hashes for legacy hardware.
	Adaptee	LegacyTurnstileSystem	Pre-existing physical security hardware requiring raw bitwise integer parameters.

4. Justification for Team-Selected Patterns

4.1 Why Strategy Pattern (Over Template Method)?

At first glance, the **Template Method** pattern appears sufficient because the initial algorithms (`FullEvacuationStrategy` and `LockdownContainmentStrategy`) exhibit structural symmetry: both manipulate the access doors first (via `DoorControlInterface`), and broadcast alerts to the mediator second (via `CampusMediator`). A Template Method base class could easily define this rigid two-step skeleton.

Why Strategy is Architecturally Superior:

- **Unpredictable Protocol Execution Order:** Emergency protocols are inherently chaotic. If the system later introduces an `ActiveShooterStrategy` , protocol safety dictates alerting law enforcement (the Mediator) *first* before taking the time to manipulate physical perimeter barriers. Conversely, a `MedicalTriageStrategy` might focus purely on paramedic deployment and never touch access doors at all.
- **Avoiding the "Rigid Skeleton" Trap:** Template Method would lock all future crisis playbooks into a rigid, unchangeable algorithm skeleton, forcing every emergency response to process doors prior to inter-unit notifications.
- **Honoring the Open/Closed Principle:** Under the **Strategy Pattern**, the Context (`ApplyStrategyCommand`) passes the necessary interfaces (`DoorControlInterface*` and `CampusMediator*`) directly to the concrete strategy and delegates complete procedural control. This grants future tactical algorithms complete architectural freedom over their internal order of execution without modifying or breaking existing base classes.

4.2 Why State Pattern (Over Conditionals and Flags)?

An emergency incident passes through distinct lifecycle phases that enforce strict business rules (e.g., an incident cannot be closed/resolved before responders have arrived and contained the threat).

- **Eliminating Giant Switch/If-Else Chains:** Without State, the `Incident` class would be polluted with complex boolean flags (`isDispatched` , `isContained` , `isResolved`) and tangled conditional logic in every single method.
- **Enforcing Architectural Invariants:** The State Pattern localizes transition rules directly inside concrete state classes (`ReportedState` , `DispatchedState` , `ContainedState` , `ResolvedState`). Illegal operations (such as resolving an un-dispatched incident) are safely intercepted and rejected at the state level without crashing the application.

4.3 Polymorphic Memory Ownership Policy

- **Command Ownership:** `OperatorConsole` manages the complete lifecycle of all executed commands, recursively deleting all `Command*` pointers stored inside its `history` vector upon shutdown.
- **State Ownership:** `Incident` cleanly manages the lifetime of its active `IncidentState*` , deleting old states upon transitions and cleaning up the final state on destruction.
- **Mediator Ownership:** `CentralDispatchMediator` maintains non-owning observer references to `ResponseUnit*` colleagues. Colleagues manage their own lifecycles, completely eliminating double-free corruption.

5. Pattern Interactivity & Architectural Boundaries

5.1 Pattern Interactivity (Where Patterns Meet)

The six patterns form a single coherent crisis response engine rather than isolated textbook examples:

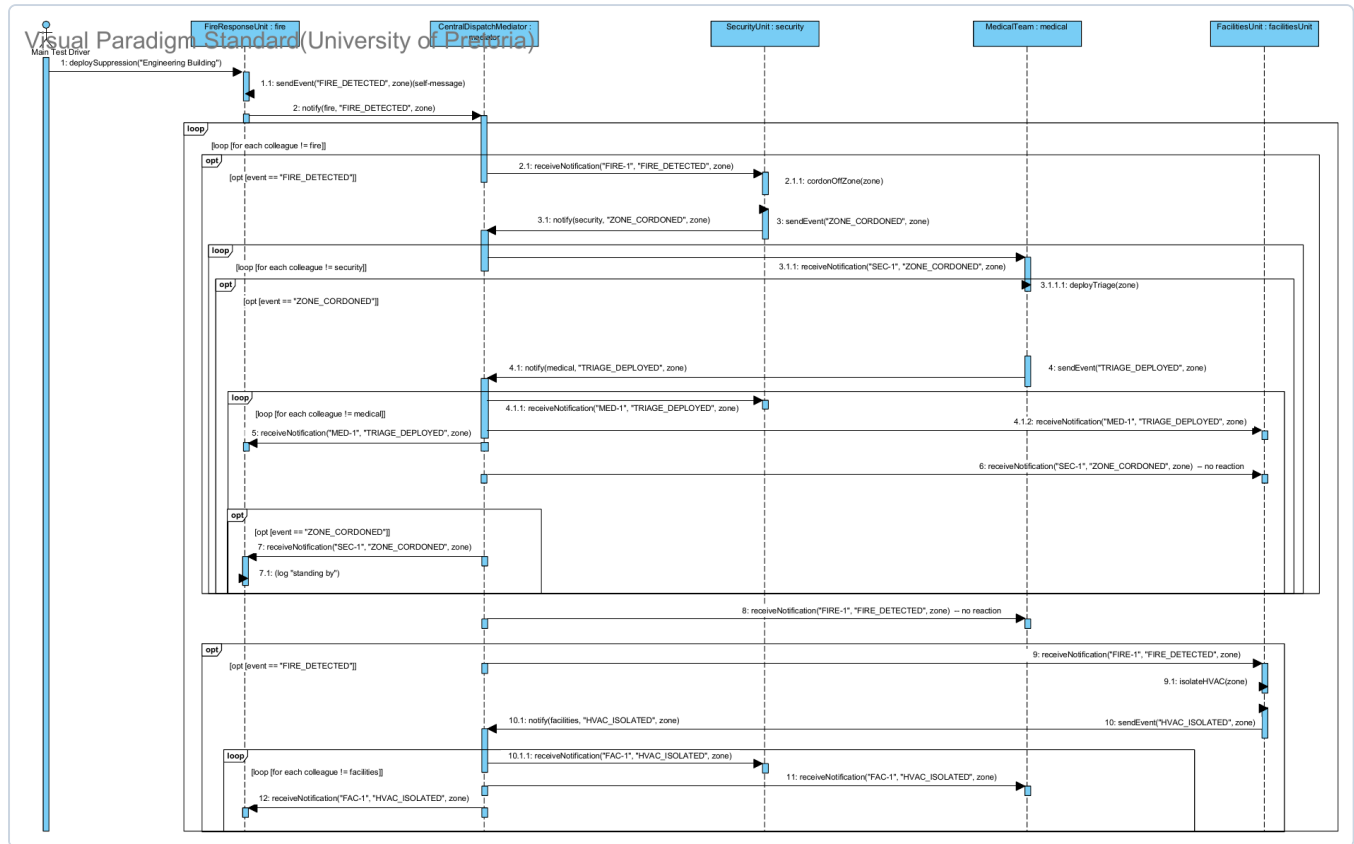
1. **Facade** → **Command**: The `CampusEmergencyFacade` coordinates multi-step emergency responses by instantiating concrete `Command` objects (`PerimeterLockCommand`, `ApplyStrategyCommand`) and passing them to the `OperatorConsole`.
2. **Command** → **Strategy**: The `ApplyStrategyCommand` serves as the execution *Context* for the `ResponseStrategy`, invoking its tactical algorithm polymorphically.
3. **Command** → **State**: The `DispatchUnitCommand` advances the target `Incident` lifecycle state upon deployment.
4. **Strategy** → **Adapter**: Tactical strategies manipulate physical zone access by issuing high-level calls to `DoorControlInterface`, which `TurnstileAdapter` translates for the legacy hardware.
5. **Strategy** → **Mediator**: Strategies broadcast crisis events (e.g. `LOCKDOWN_TRIGGERED`, `EVACUATION_ORDERED`) to `CentralDispatchMediator`, which coordinates responder actions across all departments.

5.2 Architectural Boundaries

- **Facade Boundaries**: Hides complex multi-step crisis playbooks (locking gates, dispatching teams, and applying containment atomically). Lower-level subsystems remain directly accessible: operators can issue standalone commands via `OperatorConsole`, and units can communicate directly via the mediator.
- **Adapter Boundaries**: Translates modern string zone names (e.g. "Chemistry Building") into legacy integer sector hashes and raw bit flags (`raw_set_barrier(sector=6752, lockFlag=1)`).
- **Mediator Decoupling**: Prevents an $O(N^2)$ web of direct dependencies between four departments. Colleagues hold $O(1)$ coupling through `CentralDispatchMediator`.

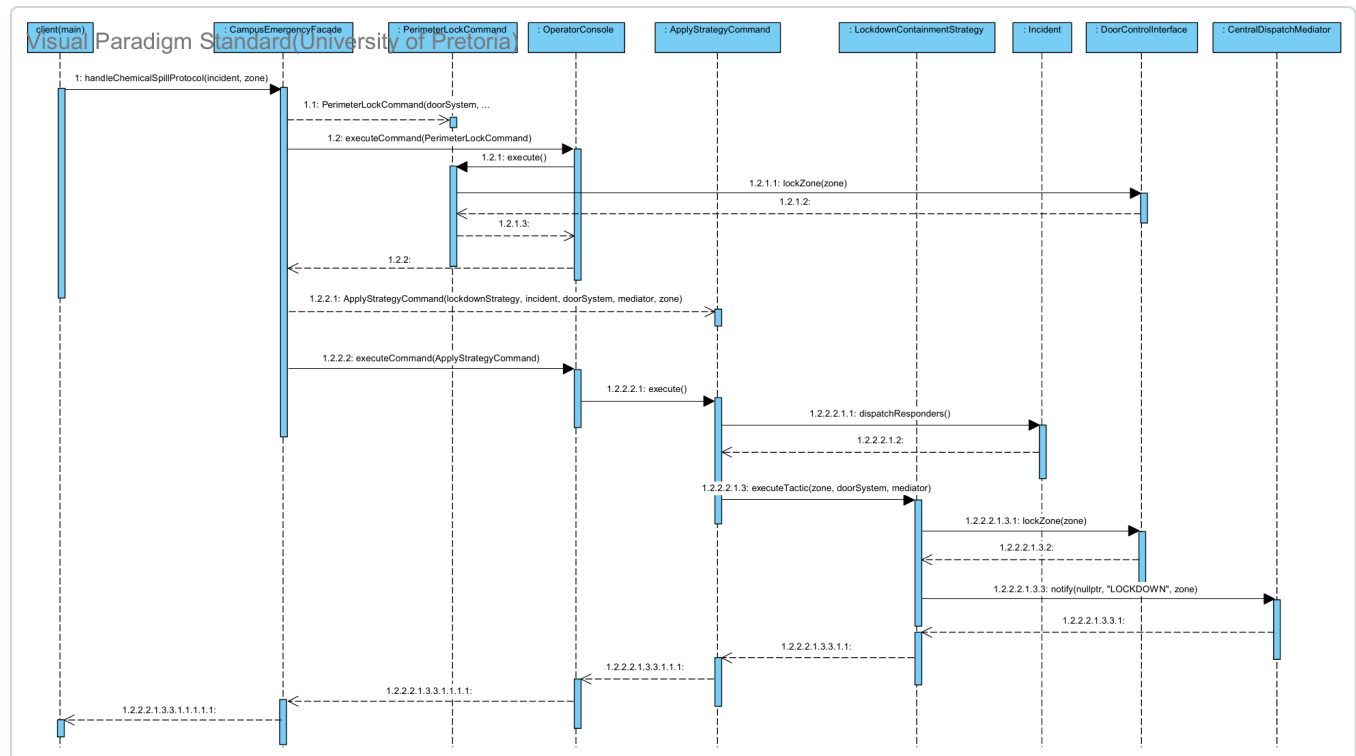
6. UML Sequence Diagram 1: Central Dispatch Mediator

Visualizes decoupled communication: `FireResponseUnit` detects a hazard, alerting the `CentralDispatchMediator`, which autonomously coordinates Security cordons, Medical triage, and Facilities HVAC isolation.



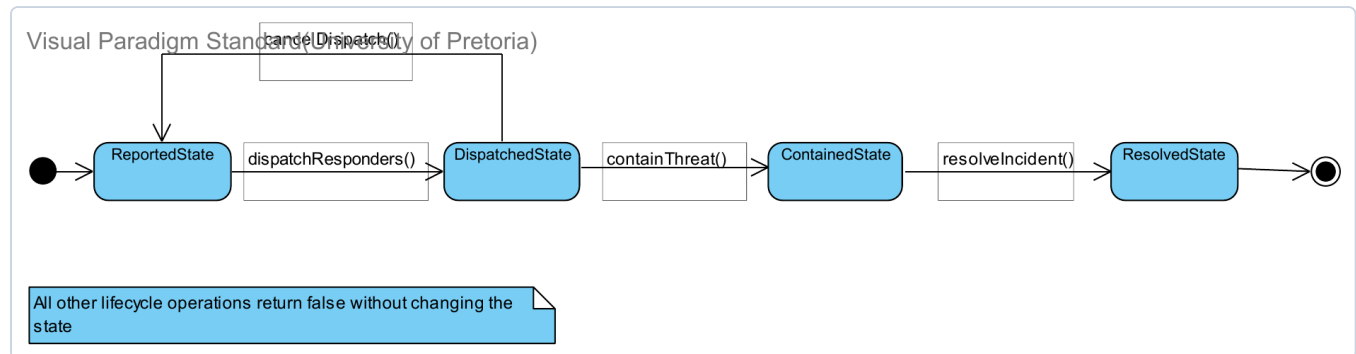
7. UML Sequence Diagram 2: Facade, Command, and Strategy Integration

Visualizes macro protocol execution: `CampusEmergencyFacade` creates and submits commands to `OperatorConsole`, which executes `ApplyStrategyCommand` and invokes tactical playbooks.



8. UML State Diagram: Incident Lifecycle State Machine

Illustrates the strict incident lifecycle state machine and valid operational transitions (`ReportedState` → `DispatchedState` → `ContainedState` → `ResolvedState`), along with protection guards enforcing valid state transitions.



9. GDB Debugging Investigation

9.1 GDB Debugging Evidence (Stepping & Inspection in Docker)

The interactive GDB session executed within the Docker container demonstrates launching the `campusguard` executable, setting a breakpoint at `main()`, stepping through execution line-by-line, and inspecting subsystem object construction:

```
edansel@EDN:/mnt/c/Users/ehome/OneDrive - acexoft/WORKSPACE (DOE)/up_school/year_two/sem2/COS214/Practicals/PRACS-COS214 (main)$ docker compose run --rm campusguard gdb ./campusguard
Container prac5-cos214-campusguard-run-729bds37fd98 Creating
Container prac5-cos214-campusguard-run-729bds37fd98 Created
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./campusguard...
(gdb)
```

```
Reading symbols from ./campusguard...
(gdb) break main
Breakpoint 1 at 0xcaf9: file src/main.cpp, line 24.
(gdb) run
Starting program: /app/campusguard
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at src/main.cpp:24
24     int main() {
(gdb) next
25         std::cout << "CampusGuard: Emergency Response Coordination System\n";
(gdb) next
CampusGuard: Emergency Response Coordination System
27         LegacyTurnstileSystem legacyHardware;
(gdb) step
LegacyTurnstileSystem::LegacyTurnstileSystem (this=0x7fffffffe9f0) at src/LegacyTurnstileSystem.h:6
6     class LegacyTurnstileSystem {
```

```
--- Incident is walked to resolution ---
[Incident #1002] Dispatched -> Contained
[Incident #1002] Contained -> Resolved
Final status: Resolved
```

```
CampusGuard shutdown complete.
[Inferior 1 (process 20) exited normally]
(gdb) quit
```

```
edansel@EDN:/mnt/c/Users/ehome/OneDrive - acexoft/WORKSPACE (DOE)/up_school/year_two/sem2/COS214/Practicals/PRACS-COS214 (main)$
```

10. Valgrind Memory Leak Verification

10.1 Automated Memory Leak Verification (Zero Leaks)

Memory verification executed directly via Docker (`docker compose run --rm campusguard valgrind --leak-check=full ./campusguard`) confirms complete memory deallocation across all design pattern subsystems with **0 errors and 0 leaks**:

```
edansel@EDN:/mnt/c/Users/ehome/OneDrive - acexoft/WORKSPACE (DOE)/up_school/year_two/sem2/COS214/Practicals/PRAC5-COS214 (main)$ docker compose run --rm campusguard valgrind --leak-check=full ./campusguard
Container prac5-cos214-campusguard-run-a6257b60e7cd Creating
Container prac5-cos214-campusguard-run-a6257b60e7cd Created
==1== Memcheck, a memory error detector
==1== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==1== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==1== Command: ./campusguard
==1==
CampusGuard: Emergency Response Coordination System

CampusGuard shutdown complete.
==1==
==1== HEAP SUMMARY:
==1==   in use at exit: 0 bytes in 0 blocks
==1==   total heap usage: 41 allocs, 41 frees, 74,674 bytes allocated
==1==
==1== All heap blocks were freed -- no leaks are possible
==1==
==1== For lists of detected and suppressed errors, rerun with: -s
==1== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
edansel@EDN:/mnt/c/Users/ehome/OneDrive - acexoft/WORKSPACE (DOE)/up_school/year_two/sem2/COS214/Practicals/PRAC5-COS214 (main)$
```

11. Containerized Demonstration & Docker Workflow

To satisfy the Task 5 and Task 6 demonstration requirements, CampusGuard is fully containerized using an Ubuntu 22.04 base image equipped with `build-essential`, `g++`, `make`, `valgrind`, and `gdb`.

11.1 Demonstration Workflow & Exact Commands

1. Compile & Launch Live Demonstration:

```
docker compose up --build
```

Compiles all C++11 source units via the Makefile, starts the container, and executes the integrated scenarios.

2. Interactive Debugging Session (GDB):

```
docker compose run --rm campusguard gdb ./campusguard
```

Runs an interactive GDB session with `SYS_PTRACE` capabilities enabled.

3. Automated Memory Leak Audit (Valgrind):

```
docker compose run --rm campusguard valgrind --leak-check=full ./campusguard
```

Runs the full scenario through Valgrind Memcheck and automatically removes the temporary container.

4. Container Teardown & Cleanup:

```
docker compose down
```

Note: Full container configurations are provided in `Dockerfile` and `docker-compose.yml` at the repository root.

11.2 GitHub Repository & Workflow Reflection

- **GitHub Repository:** <https://github.com/enockNyamweya/campusguard>
- **Team Collaboration & Branch Integration:** Work was divided by domain role and developed across isolated feature branches before continuous Pull Request integration into `main`:
 - **Command & Strategy Patterns (Enock Nyamweya / edn):** Developed on branch `command-strategy` and integrated via PR #1 (423341e). Implemented `OperatorConsole` execution and undo history (17eb3), concrete commands (ae39e7a), and response strategies (1529a11). Authored Sequence Diagram 2 (985547c), configured Docker GDB/Valgrind debugging capabilities (709e110), and performed project directory restructuring (f984c80).
 - **Mediator, Facade & Adapter Patterns (Kyle Pillay / KP23545756):** Developed on branch `mediator-facade` and integrated via PR #2 (63425a7). Implemented `CampusEmergencyFacade` (ff226a1), `CentralDispatchMediator` (6ed5534), and `ResponseUnit` subclasses (473fed7). Also contributed on branch `uml` to complete `TurnstileAdapter` (52b0803), author Sequence Diagram 1 (d75eca3), and construct the driver test cases (2d4158f).
 - **State Pattern & Lifecycle (Terrence Hluyo / Terrence):** Developed on branch `uml`. Implemented the complete `Incident` lifecycle and state transition guards across `ReportedState`, `DispatchedState`, `ContainedState`, and `ResolvedState` (commits fed5df0, 3782e67, 6b32f38, ef7f878, c8193b7, 1c55c93), and uploaded the UML State Machine Diagram (cb0a01c / StateDgm.vpp).

12. Team Contribution Statement

All team members contributed equally (33.3%) to the design, architectural specifications, implementation, and verification of the CampusGuard system:

Student Name	Student Number	Contribution %	Primary Responsibilities & Key Commits
Enock Nyamweya	u25619871	33.3%	Command Pattern (<code>OperatorConsole</code> , <code>ApplyStrategyCommand</code> , <code>DispatchUnitCommand</code> , <code>PerimeterLockCommand</code>), Strategy Pattern (<code>ResponseStrategy</code> , <code>FullEvacuationStrategy</code> , <code>LockdownContainmentStrategy</code>), Sequence Diagram 2, Master Class Diagram, Docker GDB/Valgrind configuration, and repository integration (Commits: <code>17eb3</code> , <code>ae39e7a</code> , <code>1529a11</code> , <code>985547c</code> , <code>709e110</code> , <code>f984c80</code>).
Kyle Pillay	u23545756	33.3%	Mediator Pattern (<code>CampusMediator</code> , <code>CentralDispatchMediator</code> , <code>ResponseUnit</code> hierarchy), Facade Pattern (<code>CampusEmergencyFacade</code>), Adapter Pattern (<code>TurnstileAdapter</code>), Sequence Diagram 1, and initial runtime test drivers (Commits: <code>ff226a1</code> , <code>6ed5534</code> , <code>473fed7</code> , <code>52b0803</code> , <code>d75eca3</code> , <code>2d4158f</code>).
Terrence Hluyo	u25446968	33.3%	State Pattern implementation (<code>Incident</code> , <code>IncidentState</code> , <code>ReportedState</code> , <code>DispatchedState</code> , <code>ContainedState</code> , <code>ResolvedState</code>), cancellation handler logic, and the UML State Machine Diagram (Commits: <code>fed5df0</code> , <code>3782e67</code> , <code>6b32f38</code> , <code>ef7f878</code> , <code>c8193b7</code> , <code>1c55c93</code> , <code>cb0a01c</code>).